

Preprocessing_Features_iparlato

November 30, 2025

1 Preprocessing & Feature Engineering Report for Shark Movement Forecasting: An Assessment for Ocean Recreational Safety

By: Isabella Parlato Github Repository: https://github.com/ParlatoI/Shark-Forecasting_Capstone

First, we'll need to bring in the focus datasets. From the Github repository in the "raw_datasets" folder, bring the following datasets into the colab content folder: "buoy_41009_clean" and "Regularized_tiger_shark_tracks"

For now, I have decided to just proceed with the one buoy dataset for Buoy 41009 since that's the one nearest/more relevant to the locations of the sharks in that dataset. Buoy 41010 was a bit further out and in the EDA report, both buoys exhibited the same patterns with only a slight difference between them in variables typically. Buoy 41009 is also the buoy that's closer to the coastline.

```
[44]: # initial loading of libraries & pulling in datasets

import pandas as pd
import numpy as np
import sys
import matplotlib.pyplot as plt
from datetime import datetime

# for preprocessing
from sklearn.model_selection import train_test_split, TimeSeriesSplit
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# for unsupervised methods
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import cdist
```

```
[1]: # function for determining ideal split-ratio for train-test sets

import sys

## Check if packages are loaded; if not, import them
if 'numpy' not in sys.modules:
    import numpy as np
if 'pandas' not in sys.modules:
    import pandas as pd

def calcSplitRatio(df, p=None):
    """
    Calculate the ideal split ratio for training and testing sets.

    Parameters:
    p (int): The number of parameters (optional). If not provided, defaults to
    the number of columns minus 1 (assuming the last column is the target).

    df (DataFrame): The DataFrame that will be used for analysis.

    Returns:
    float: The training set proportion.
    """

    ## Set p to the number of predictors if not provided
    if p is None:
        p = df.shape[1] - 1 ## The number of features (predictors),
                           ## assuming last column is target

    ## Calculate ideal number for testing set
    test_N = (1 / np.sqrt(p)) * len(df)

    ## Calculate testing proportion
    test_prop = round(test_N / len(df), 2)

    ## Calculate training proportion
    train_prop = 1 - test_prop

    ## Print the results
    print(f"The ideal split ratio is {train_prop}:{test_prop} training:testing")

    ## Return the size of the training set proportion
    return train_prop
```

```
[9]: # load datasets in & confirm column names

# load buoy and shark tracking datasets
```

```

buoy = pd.read_csv("/content/buoy_41009_clean.csv")
sharks = pd.read_csv("/content/Regularized_tiger_shark_tracks.csv")

# standardize col. names to lowercase
buoy.columns = buoy.columns.str.lower().str.strip()
sharks.columns = sharks.columns.str.lower().str.strip()

print("Buoy columns:", buoy.columns.tolist())
print("Shark columns:", sharks.columns.tolist())

```

Buoy columns: ['year', 'month', 'day', 'hour', 'minute', 'wdir', 'wspd', 'gst', 'wvht', 'dpd', 'apd', 'mwd', 'pres', 'atmp', 'wtmp', 'dewp', 'datetime']
Shark columns: ['obs', 'shark_id', 'lon', 'lat', 'month', 'day', 'year', 'season']

```

[10]: # create new clean datetime column
buoy["timestamp"] = pd.to_datetime(buoy["datetime"], errors="coerce")
## errors should handle with NaT (Not a Time) for missing

# create timestamp to add to month-day-year for shark dataset since buoy dataset
↳ has times
sharks["timestamp"] = pd.to_datetime(
    sharks[["year", "month", "day"]].astype(str).agg("-".join, axis=1),
    format="%Y-%m-%d", ## in string, date as 4 digit year, 2 digit month, 2
    ↳ digit day format
    errors="coerce"
)

# default time assigned is 12:00 for mid-day for shark dataset which lacks
↳ hourly times
sharks["timestamp"] = sharks["timestamp"] + pd.Timedelta("12:00:00")

```

Since this project isn't looking into time of days with variations, more weather by the day with shark presence, creating standard timestamp to allow merge between datasets. So need to keep in mind that this "mid-day" assumption is being made.

```

[16]: # merge shark detections with buoy conditions dataset on nearest timestamp

sharks = sharks.sort_values("timestamp")
buoy = buoy.sort_values("timestamp")

merged = pd.merge_asof(
    sharks,
    buoy,
    on="timestamp",
    direction="nearest",
    tolerance=pd.Timedelta("24h") # daily date for full day window
)

```

```

)

print("Merged dataset columns:", merged.columns.tolist())
print("Merged shape before filtering:", merged.shape)

# Remove detections with no matching buoy record
merged = merged.dropna(subset=["wspd"])

print("Merged shape after filtering:", merged.shape)

unmatched_dates = sharks[~sharks["timestamp"].
    ↪isin(merged["timestamp"])]["timestamp"]
print(unmatched_dates.head())
print("Total unmatched:", len(unmatched_dates))
unmatched_dates.dt.date.value_counts().sort_index().plot(kind="bar",
    ↪figsize=(10,4))

```

```

Merged dataset columns: ['obs', 'shark_id', 'lon', 'lat', 'month_x', 'day_x',
'year_x', 'season', 'timestamp', 'year_y', 'month_y', 'day_y', 'hour', 'minute',
'wdir', 'wspd', 'gst', 'wvht', 'dpd', 'apd', 'mwd', 'pres', 'atmp', 'wtmp',
'dewp', 'datetime']

```

```

Merged shape before filtering: (5227, 26)

```

```

Merged shape after filtering: (5182, 26)

```

```

1321    2012-08-05 12:00:00

```

```

1322    2012-08-06 12:00:00

```

```

1323    2012-08-07 12:00:00

```

```

1324    2012-08-08 12:00:00

```

```

1325    2012-08-09 12:00:00

```

```

Name: timestamp, dtype: datetime64[ns]

```

```

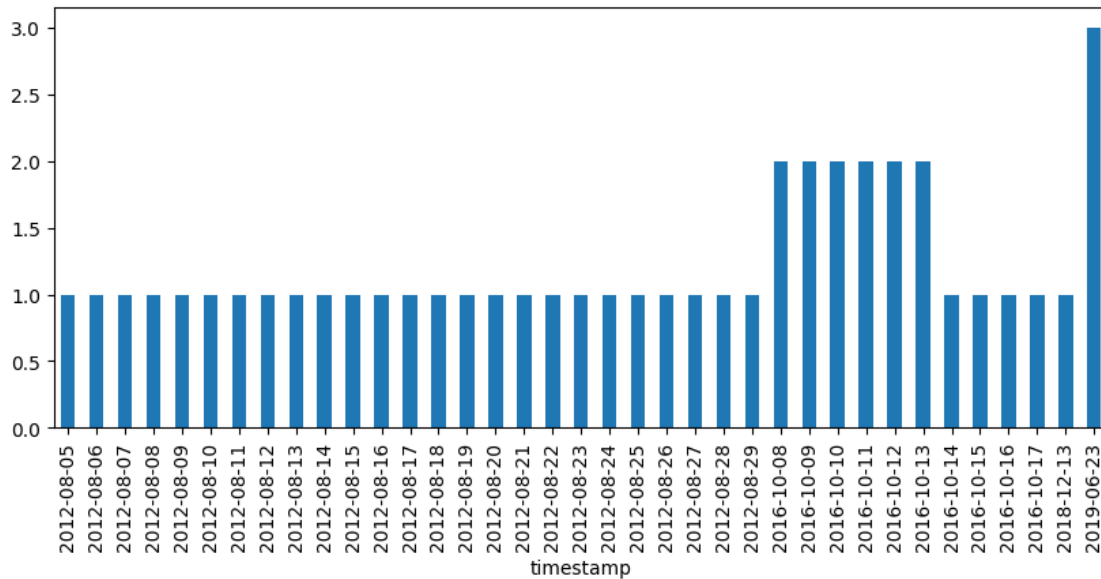
Total unmatched: 45

```

```

[16]: <Axes: xlabel='timestamp'>

```



I needed a way to connect the more irregular shark movement detections with regularly sampled environmental data so I did a daily tolerance for lining up nearest timestamps. Ultimately, decided to drop rows where no environmental buoy data connected since we need that context to be able to use them for modeling. It was an extremely small loss of only 45 rows.

August 5, 2012 through August 29, 2012, October 8, 2016 through October 17, 2016, December 13, 2018, and June 23, 2019 - plot shows the groupings. Likely buoy outages or issues in collecting data until it was able to be fixed.

```
[26]: # create shark presence/absence response variable - binary
      ## using background samples

      # Presence rows
      merged["presence"] = 1

      # Absence rows: every buoy timestamp, labeled 0
      background = pd.DataFrame({"timestamp": buoy["timestamp"]})
      background["presence"] = 0

      absence_df = pd.merge_asof(
          background.sort_values("timestamp"),
          buoy.sort_values("timestamp"),
          on="timestamp",
          direction="nearest"
      )

      # Remove timestamps that coincide with shark presence
```

```

## would confuse the model if the sharks were both present and absent on same
    ↪ day
absence_df = absence_df[~absence_df["timestamp"].isin(merged["timestamp"])]

print("Presence timestamps:", len(merged["timestamp"].unique()))
print("Absence timestamps before filtering:", len(background))
print("Absence timestamps after filtering:", len(absence_df))

```

```

Presence timestamps: 2136
Absence timestamps before filtering: 253464
Absence timestamps after filtering: 252758

```

Since the tiger shark dataset only contains shark location detections, I needed some way to create “absence” data as well. So going to do so using the buoy’s timeline/data samples for dates to create background absence variable. It was important to then clean the dataset of any rows that would be contradictory so absence rows should be times that the buoy took in environmental readings but sharks were not detected. It looks like 706 rows were removed that were contradictory.

It makes sense that absence rows initially had about high missingness for what would be missing Shark related data like shark ID, coordinates, shark day/month/year & season of detection. I don’t want to lose coordinates of the sharks so for any absence columns, I’ll just use the buoy’s location

https://noaa-mirror.org/www.ndbc.noaa.gov/station_page.php@station=41009

Owned and maintained by National Data Buoy Center 3-meter discus buoy SCOOP payload 28.508 N 80.185 W (28°30’27” N 80°11’6” W)

Site elevation: sea level Air temp height: 3.4 m above site elevation Anemometer height: 3.8 m above site elevation Barometer elevation: 2.4 m above mean sea level Sea temp depth: 2 m below water line Water depth: 42 m Watch circle radius: 115 yards

```

[35]: # add in buoy location: Latitude: 28.508° N, Longitude: -80.185° W to Absence
    ↪ Rows

buoy_lat = 28.508
buoy_lon = -80.185

absence_df["lat"] = buoy_lat
absence_df["lon"] = buoy_lon

# if leftover lat/lon-like columns exist, remove them
for col in ["lat_x", "lat_y", "latitude", "lon_x", "lon_y", "longitude"]:
    if col in absence_df.columns:
        absence_df = absence_df.drop(columns=[col])
    if col in merged.columns:
        merged = merged.drop(columns=[col])

# combine into full dataset

```

```
full_df = pd.concat([merged, absence_df], ignore_index=True).
    ↪sort_values("timestamp")
print("Full dataset shape:", full_df.shape)
```

Full dataset shape: (257940, 30)

[36]: *# explore missingness*

```
missing_summary = full_df.isnull().mean().sort_values(ascending=False)
missing_summary
```

```
[36]: obs          0.979910
      shark_id     0.979910
      day_x        0.979910
      month_x      0.979910
      season       0.979910
      year_x       0.979910
      day_y        0.979910
      month_y      0.979910
      year_y       0.979910
      mwd          0.770823
      dpd          0.594979
      wvht         0.594348
      apd          0.594348
      wtmp         0.179495
      dewp         0.027367
      day          0.020090
      year         0.020090
      month        0.020090
      pres         0.017376
      atmp         0.006215
      wdir         0.003691
      gst          0.000341
      wspd         0.000279
      minute       0.000000
      timestamp    0.000000
      lon          0.000000
      lat          0.000000
      hour         0.000000
      datetime     0.000000
      presence     0.000000
      dtype: float64
```

Reminder of results notes from EDA correlation heatmap for the buoy variables:

1. GST & WSPD are 0.99 - plan to drop one
2. Second highest at 0.89 correlation is DEWP & ATMP - plan to drop one
3. followed by ATMP & WTMP at 0.82 - plan to drop one

4. WTMP & DEWP followed at 0.67 correlation
5. GST & WVHT at 0.66
6. WVHT & WSPD at 0.63
7. DPD & APD at 0.62

```
[37]: # remove top 3 high correlation environmental variables

corr_pairs = [
    ("gst", "wspd"),      # 0.99 correlation
    ("dewp", "atmp"),     # 0.89 correlation
    ("atmp", "wtmp")      # 0.82 correlation
]

# the goal is to drop one of the correlated pair with more missingness than its
# partner variable
columns_to_drop_corr = []

for col1, col2 in corr_pairs:
    if col1 in full_df.columns and col2 in full_df.columns:
        miss1 = full_df[col1].isnull().mean()
        miss2 = full_df[col2].isnull().mean()

        if miss1 > miss2:
            drop_col = col1
        else:
            drop_col = col2

        columns_to_drop_corr.append(drop_col)
        print(f"Dropping '{drop_col}' due to high correlation between {col1}
        and {col2}.")
    else:
        print(f"Skipping pair ({col1}, {col2}) - one or both columns missing.")

full_df = full_df.drop(columns=columns_to_drop_corr)
print("\nDropped due to correlation:", columns_to_drop_corr)
```

Dropping 'gst' due to high correlation between gst and wspd.
 Dropping 'dewp' due to high correlation between dewp and atmp.
 Dropping 'wtmp' due to high correlation between atmp and wtmp.

Dropped due to correlation: ['gst', 'dewp', 'wtmp']

```
[38]: # now to drop shark only features that aren't needed

shark_only_cols = [
    "obs", "shark_id", "month_x", "day_x", "year_x", "season",
    "year_y", "month_y", "day_y"
```

```

]

full_df = full_df.drop(columns=[c for c in shark_only_cols if c in full_df.
    ↪columns])

# Check missingness again
missing_summary = full_df.isnull().mean().sort_values(ascending=False)
missing_summary

```

```

[38]: mwd          0.770823
      dpd          0.594979
      apd          0.594348
      wvht         0.594348
      year         0.020090
      month        0.020090
      day          0.020090
      pres         0.017376
      atmp         0.006215
      wdir         0.003691
      wspd         0.000279
      lon          0.000000
      lat          0.000000
      minute       0.000000
      hour         0.000000
      timestamp    0.000000
      datetime     0.000000
      presence     0.000000
      dtype: float64

```

In running new missingness summary, there were 4 variables with missingness over about 50%:

1. mwd (mean wave direction) - about 77%
2. dpd (dominant wave period) - about 59%
3. wvht (wave height) - about 59%
4. apd (average wave period) - about 59%

everything else has way less missingness (about 2% or lower) so planning to drop these because otherwise I may need to impute them, which might add noise, and detract from the concrete actual environmental data

```

[39]: # drop variables with missingness over 50%

missing_summary = full_df.isnull().mean()
high_missing_cols = missing_summary[missing_summary > 0.50].index.tolist()

full_df = full_df.drop(columns=high_missing_cols)

print("Dropped for missingness:", high_missing_cols)

```

Dropped for missingness: ['wvht', 'dpd', 'apd', 'mwd']

```
[40]: # Save cleaned dataset as new csv

full_df.to_csv("cleaned_merged_shark_buoy.csv", index=False)
```

This is a good point to pull the dataset into a new csv since it's before any splitting for the models or model-type specific focused preprocessing.

2 Split into train & test sets - model specific preprocessing

```
[41]: # use ideal split ratio function

# calc. recommended split ratio
recommended_train_ratio = calcSplitRatio(full_df)

# convert to split index
split_idx = int(len(full_df) * recommended_train_ratio)
```

The ideal split ratio is 0.72:0.28 training:testing

```
[42]: # time-aware train/test split
## need to avoid future info. leaking into the past (data leakage)

# sort by time before splitting (VERY important)
full_df = full_df.sort_values("timestamp")

# time-aware split
train_df = full_df.iloc[:split_idx]
test_df = full_df.iloc[split_idx:]

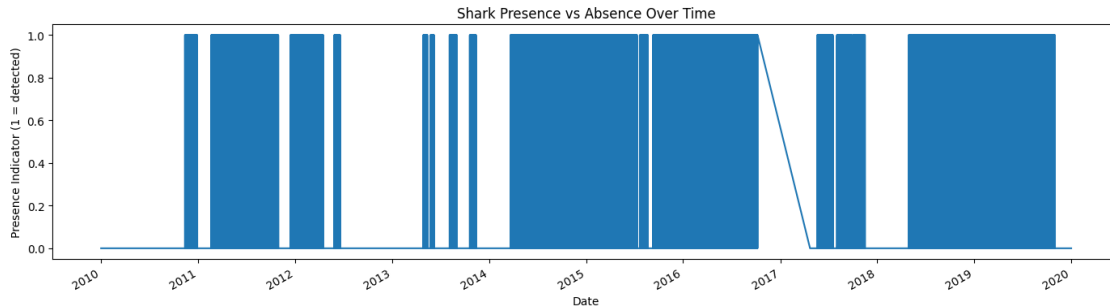
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
```

Train shape: (185716, 14)

Test shape: (72224, 14)

```
[45]: # presence vs absence over time plot

plt.figure(figsize=(14,4))
full_df.groupby("timestamp")["presence"].mean().plot()
plt.title("Shark Presence vs Absence Over Time")
plt.xlabel("Date")
plt.ylabel("Presence Indicator (1 = detected)")
plt.tight_layout()
plt.show()
```



There's definitely periods of time where the shark detections were lumped together

```
[48]: # supervised preprocessing pipeline

# split predictors & target
X_train = train_df.drop(columns=["presence"])
y_train = train_df["presence"]

X_test = test_df.drop(columns=["presence"])
y_test = test_df["presence"]

# id numeric & categorical features
numeric_features = X_train.select_dtypes(include=["float64", "int64"]).columns
categorical_features = X_train.select_dtypes(include=["object"]).columns

# set preprocessing for each type
numeric_pipe = Pipeline([("scale", StandardScaler())])
categorical_pipe = Pipeline([("encode",
    ↳ OneHotEncoder(handle_unknown="ignore"))])

# build final supervised preprocessor
supervised_preprocessor = ColumnTransformer(
    [
        ("num", numeric_pipe, numeric_features),
        ("cat", categorical_pipe, categorical_features)
    ]
)

print("Numeric features:", numeric_features.tolist())
print("Categorical features:", categorical_features.tolist())
```

```
Numeric features: ['lon', 'lat', 'hour', 'minute', 'wdir', 'wspd', 'pres',
'atmp', 'year', 'month', 'day']
```

```
Categorical features: ['datetime']
```

Supervised models need standardized numeric inputs to keep equal contribution and encoded (one-

hot) categorical inputs.

Potential models: logistic regression (baseline), random forest, gradient boosting, and/or LNN

```
[53]: # unsupervised preprocessing pipeline

from sklearn.impute import SimpleImputer

# remove target to avoid leakage & known non-num. col. for unsup. modeling
unsup_features = train_df.drop(columns=["presence", "timestamp", "datetime"])

# select ONLY num. columns (trying to avoid accidental strings)
unsup_features = unsup_features.select_dtypes(include=["float64", "int64"])

# impute missing values - K-means can't handle NaN
imputer = SimpleImputer(strategy="mean")
unsup_imputed_train = imputer.fit_transform(unsup_features)

unsup_imputed_test = imputer.transform(
    test_df[unsup_features.columns]
)

# Build scaler
scaler = StandardScaler()

# Fit-transform for training
X_unsup_train = scaler.fit_transform(unsup_imputed_train)

# Transform for test
X_unsup_test = scaler.transform(unsup_imputed_test)

print("Unsupervised feature matrix shape:", X_unsup_train.shape)
print("Remaining NaNs in unsupervised matrix:", np.isnan(X_unsup_train).any())
print("Columns used:", unsup_features.columns.tolist())
```

Unsupervised feature matrix shape: (185716, 11)

Remaining NaNs in unsupervised matrix: False

Columns used: ['lon', 'lat', 'hour', 'minute', 'wdir', 'wspd', 'pres', 'atmp', 'year', 'month', 'day']

Unsupervised models must operate on purely numeric variables because clustering can only look at numbers so only keeping so non-numeric fields removed.

Potential/planned models: K-Means Clustering, Hierarchical Clustering & PCA

3 Unsupervised Models

```
[55]: # K-Means Clustering

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_unsup_train)

train_df.loc[:, "kmeans_cluster"] = kmeans.labels_
test_df.loc[:, "kmeans_cluster"] = kmeans.predict(X_unsup_test)
## .loc to clear up pd warnings

print("K-means cluster counts:", pd.Series(kmeans.labels_).value_counts())
```

```
K-means cluster counts: 1    73824
2    64532
0    47360
Name: count, dtype: int64
```

Seems to have worked okay so we ended up with three clusters as intended.

```
[ ]: # Hierarchical Clustering

# ward linkage (minimizes within-cluster variance)
Z = linkage(X_unsup_train, method="ward")

# assign 3 clusters
hier_train = fcluster(Z, t=3, criterion="maxclust")
train_df.loc[:, "hier_cluster"] = hier_train

# compute centroids for test-set assignment
centroids = np.vstack([
    X_unsup_train[hier_train == k].mean(axis=0)
    for k in np.unique(hier_train)
])

# assign nearest-centroid cluster on test set
test_df.loc[:, "hier_cluster"] = np.argmin(
    cdist(X_unsup_test, centroids), axis=1
) + 1

print("Hierarchical cluster counts:")
print(pd.Series(hier_train).value_counts())
```

no built-in prediction method so test-set clusters are assigned using their nearest centroid mapping group

```
[56]: # PCA
```

```

# fit PCA on training unsup features
pca = PCA(n_components=5)
pca.fit(X_unsup_train)

# transform train & test sets
train_pca = pca.transform(X_unsup_train)
test_pca = pca.transform(X_unsup_test)

print("Explained variance ratios:", pca.explained_variance_ratio_)

```

Explained variance ratios: [0.16348428 0.15604876 0.11714706 0.10358938 0.09439429]

This tells us the total variation of environmental conditions. Seems like there is no single factor dominating these variables

Note to self: plan to do/further review Time-series lag feature creation

```

[59]: # new data dictionary

def create_data_dictionary(df):
    """
    Generate a data dictionary including column name,
    data type, percent missing, and example values.
    """

    data_dict = pd.DataFrame({
        "column_name": df.columns,
        "data_type": df.dtypes.astype(str),
        "percent_missing": df.isna().mean().round(4),
        "example_value": df.apply(lambda col: col.dropna().iloc[0] if col.
↳ dropna().size > 0 else None)
    })

    return data_dict

# Generate dictionary
data_dictionary = create_data_dictionary(full_df)

# Display it
data_dictionary

```

```

[59]:
      column_name  data_type  percent_missing  example_value
lon           lon   float64           0.0000          -80.185
lat           lat   float64           0.0000           28.508
timestamp  timestamp  datetime64[ns]       0.0000  2010-01-01 19:20:00
hour          hour   float64           0.0000           19.0

```

minute	minute	float64	0.0000	20.0
wdir	wdir	float64	0.0037	328.0
wspd	wspd	float64	0.0003	10.1
pres	pres	float64	0.0174	1013.1
atmp	atmp	float64	0.0062	17.1
datetime	datetime	object	0.0000	2010-01-01 19:20:00
presence	presence	int64	0.0000	0
year	year	float64	0.0201	2010.0
month	month	float64	0.0201	1.0
day	day	float64	0.0201	1.0

[60]: *# first download dataset from colab as ipynb then reupload to content folder*
now to convert to pdf

```
!sudo apt-get install texlive-xetex texlive-fonts-recommended_
↪texlive-plain-generic
!sudo apt-get update
!sudo apt-get install pandoc
!jupyter nbconvert --to pdf /content/Preprocessing-Features_iparlato.ipynb
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-plain-generic is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 https://cli.github.com/packages stable InRelease
Hit:3 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
InRelease
Hit:4 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:9 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:10 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy
InRelease
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

pandoc is already the newest version (2.9.2.1-3ubuntu2).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
[NbConvertApp] WARNING | pattern '/content/Preprocessing-
Features_iparlato.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y

Answer yes to any questions instead of prompting.

Equivalent to: [--JupyterApp.answer_yes=True]

--execute

Execute the notebook prior to export.

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace

Run nbconvert in place, overwriting the existing notebook (only

relevant when converting to notebook format)

Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
 --clear-output

Clear output of current file and save in place,
 overwriting the existing notebook.

Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=
 --ClearOutputPreprocessor.enabled=True]
 --coalesce-streams

Coalesce consecutive stdout and stderr outputs into one stream (within each
 cell).

Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=
 --CoalesceStreamsPreprocessor.enabled=True]
 --no-prompt

Exclude input and output prompts from converted document.

Equivalent to: [--TemplateExporter.exclude_input_prompt=True
 --TemplateExporter.exclude_output_prompt=True]
 --no-input

Exclude input cells and output prompts from converted document.

This mode is ideal for generating code-free reports.

Equivalent to: [--TemplateExporter.exclude_output_prompt=True
 --TemplateExporter.exclude_input=True
 --TemplateExporter.exclude_input_prompt=True]
 --allow-chromium-download

Whether to allow downloading chromium if no suitable version is found on the
 system.

Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
 --disable-chromium-sandbox

Disable chromium security sandbox when converting to PDF..

Equivalent to: [--WebPDFExporter.disable_sandbox=True]
 --show-input

Shows code input. This flag is only useful for dejavu users.

Equivalent to: [--TemplateExporter.exclude_input=False]
 --embed-images

Embed the images as base64 dataurls in the output. This flag is only useful
 for the HTML/WebPDF/Slides exports.

Equivalent to: [--HTMLExporter.embed_images=True]
 --sanitize-html

Whether the HTML in Markdown cells and cell outputs should be sanitized..

Equivalent to: [--HTMLExporter.sanitize_html=True]
 --log-level=<Enum>

Set the log level by value or name.

Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
 'CRITICAL']

Default: 30

Equivalent to: [--Application.log_level]

```

--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
        ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
        or a dotted object name that represents the import path for an
        ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
        Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                to output to the directory of each notebook.

```

To recover
current
previous default behaviour (outputting to the
working directory) use . as the flag value.

Default: ''
Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
The URL prefix for reveal.js (version 3.x).
This defaults to the reveal CDN, but can be any url pointing to a
copy
of reveal.js.
For speaker notes to work, this must be a relative path to a local
copy of reveal.js: e.g., "reveal.js".
If a relative path is given, it must be a subdirectory of the
current directory (from which the server is run).
See the usage documentation
([https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow](https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow))
for more details.

Default: ''
Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
The nbformat version to write.
Use this to downgrade notebooks.
Choices: any of [1, 2, 3, 4]
Default: 4
Equivalent to: [--NotebookExporter.nbformat_version]

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX
includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and
'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
```

```
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.